Lab 03: Detection Tuning & Rule Validation

An advanced exercise in understanding the recall / false-positive trade-off

Learning goals

- Run a narrow rule against a known, versioned set of synthetic cases.
- Explain how a rule can miss a process variant and trigger on legitimate activity.
- Make one controlled rule change and compare the resulting metrics.
- Write a compact detection specification that another learner can reproduce.

SCOPE This exercise uses synthetic CSV records and redacted placeholders. Do not run placeholder command lines. No malware, live scanning, or cloud resources are required.

Part A — Capture the baseline

From the website folder, run this command in PowerShell or a terminal:python public/labs/evaluate-detection-cases.py public/labs/detection-evaluation-cases.csv

Record the TP, FP, TN, FN counts and the metrics printed by the script. The script implements an intentionally narrow teaching rule: process name must be powershell.exe and the command line must include an -enc or -encodedcommand argument token.

BASELINE HYPOTHESIS The narrow rule should miss the synthetic pwsh.exe case, and it should flag the labelled benign encoded-maintenance case. That gives us both a false negative and a false positive to study.

Part B — Inspect the edge cases

Case	What happens in baseline	Question
case-007	powershell.exe with encoded marker; labelled benign; rule alerts	What context is needed before deciding whether this is malicious?
case-008	pwsh.exe with encoded marker; labelled suspicious; rule misses it	Which process-name assumption caused the false negative?
case-009	pwsh.exe with encoded marker; labelled benign; rule does not alert in baseline	What may happen to false positives if pwsh.exe is added to the rule?

Part C — Make one change

In a local copy of public/labs/evaluate-detection-cases.py, expand the process-name condition to include pwsh.exe as well as powershell.exe. Do not change any labels in the CSV. Run the same command again. Keep the original script so you can compare the baseline and revised behavior.

1. Confirm case-008 now alerts.
2. Confirm case-009 now alerts too, creating a false positive against its benign label.
3. Compare recall, precision, specificity, and F1 between the baseline and revised version.
4. Write down which metric improved, which worsened, and why.
5. Revert the change or save it as a separate version and record the rule version in your report.

Discussion: expected trade-off

Metric	Narrow baseline	With pwsh.exe added
TP / FP / TN / FN	2 / 1 / 4 / 1	3 / 2 / 3 / 0
Recall	0.667	1.000
Precision	0.667	0.600
Specificity	0.800	0.600
F1	0.667	0.750

These are the expected results for this deliberately constructed training corpus. They illustrate a rule trade-off, not the accuracy expected in a real environment. The CSV labels are manually assigned and the dataset is tiny.

Part D — Write a detection specification

- Rule ID and version.
- Purpose and observable behavior being detected.
- Required log source and exact fields.
- Detection logic and process-name normalization assumptions.
- Known false positives and likely blind spots.
- Test corpus version, counts, metrics, and exclusions.
- Recommended analyst checks and what cannot be concluded from the alert alone.

ADVANCED HABIT Do not optimize for a single metric in isolation. Select trade-offs using the use case, log quality, investigation capacity, and a representative evaluation set.

Further reading

- Sigma rule repository
- Sigma specification
- MITRE ATT&CK